

Model Binding using FromQuery in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Model Binding using FromQuery in ASP.NET Core Web API

In this article, I will discuss How to Implement **Model Binding Using FromQuery in ASP.NET Core Web API** Application with Examples. Please read our previous article discussing [Model Binding Using FromForm in ASP.NET Core Web API](#) with Examples.

What is a Query String in ASP.NET Core Web API?

A query string is a part of a URL that contains data to be passed to web applications. It is typically used to send parameters and values from a client to a server in HTTP requests. The query string appears after the question mark (?) in a URL and consists of key-value pairs, where each key is separated from its value by an equal sign (=). Multiple key-value pairs are separated by an ampersand (&). For example, in the URL **https://example.com/api/values?Department=IT&Gender=Male**, the query string is **Department=IT&Gender=Male**, with **Department** and **Gender** being the keys and **IT** and **Male** their respective values.

Query strings are used to pass data to the server as part of the URL, making them highly useful in HTTP GET requests where data needs to be retrievable without altering the state of the server.

Model Binding using FromQuery Attribute in ASP.NET Core Web API

The **FromQuery** attribute in ASP.NET Core Web API is used to bind a parameter in an action method to a value provided in the query string of the HTTP request. This attribute is useful when we want to explicitly specify that a certain parameter should be populated from the query string. Query strings are the part of the URL that comes after the “?” character and typically include key-value pairs.

How Does the FromQuery Attribute Work in ASP.NET Core Web API?

When a request is made to a Web API, the **FromQuery** attribute tells the model binding system to populate the values of the parameters annotated with this attribute from the HTTP query string. For example, if your URL is **https://example.com/api/values?name=John&age=30**, the **FromQuery** attribute will map the **name** and **age** from the query string to the corresponding action parameters.

```
[HttpGet]
public IActionResult GetPerson([FromQuery] string name, [FromQuery] int age)
```

```
{  
    return Ok(new { Name = name, Age = age });  
}
```

So, when the `FromQuery` attribute is applied to a parameter in an action method, the model binder in ASP.NET Core looks for a value with a matching key in the query string of the request URL. If a match is found, the value is automatically converted to the type of the parameter and passed to the action method.

Example to Understand FromQuery Attribute in ASP.NET Core Web API:

Let us see an example of the `FromQuery` attribute in ASP.NET Core Web API. First, modify the `UserModel` class as follows, which will hold the User data.

```
namespace ModelBinding.Models  
{  
    public class UserModel  
    {  
        public int Id { get; set; }  
        public string Name { get; set; }  
        public string Department { get; set; }  
        public string Gender { get; set; }  
        public int Salary { get; set; }  
    }  
}
```

Using FromQuery Attribute in Controller Action Method:

To use `FromQuery`, we need to decorate the parameters of the action method with the `[FromQuery]` attribute. This tells ASP.NET Core to look for values with matching names in the query string and bind them to the parameters. For a better understanding, please modify the User Controller as follows.

```
using Microsoft.AspNetCore.Mvc;  
using ModelBinding.Models;  
namespace ModelBinding.Controllers  
{  
    [Route("api/[controller]")]  
    [ApiController]  
    public class UserController : ControllerBase  
    {  
        private static List<UserModel> Users = new List<UserModel>  
        {  
            new UserModel { Id = 1, Name = "Rakesh", Department = "IT", Gender =  
"Male", Salary = 1000 },  
            new UserModel { Id = 2, Name = "Priyanka", Department = "IT", Gender =  
"Female", Salary = 2000 },  
        }  
    }  
}
```

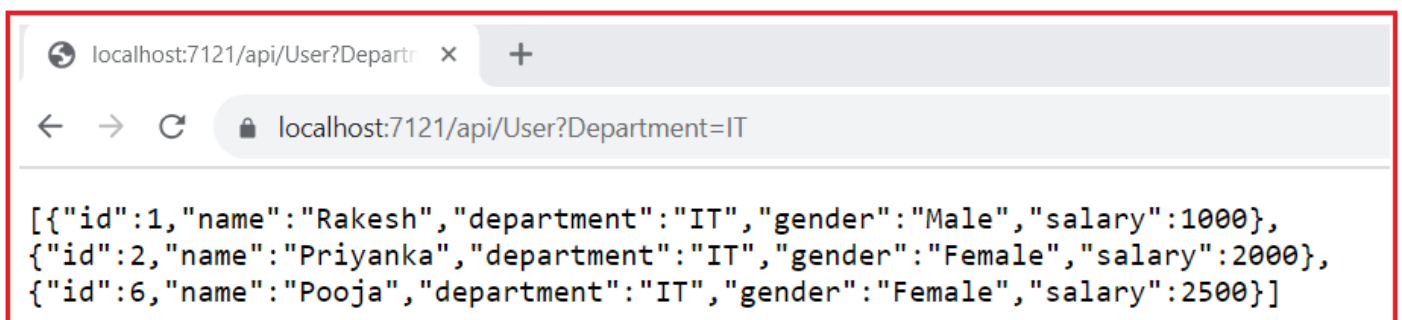
```

        new UserModel { Id = 3, Name = "Suresh", Department = "HR", Gender =
"Male", Salary = 3000 },
        new UserModel { Id = 4, Name = "Hina", Department = "HR", Gender =
"Female", Salary = 4000 },
        new UserModel { Id = 5, Name = "Pranaya", Department = "HR", Gender =
"Male", Salary = 35000 },
        new UserModel { Id = 6, Name = "Pooja", Department = "IT", Gender =
"Female", Salary = 2500 },
    };

    [HttpGet]
    public IActionResult GetProducts([FromQuery] string Department)
    {
        // Implementation to retrieve employees based on the Department
        var FilteredUsers = Users.Where(emp =>
emp.Department.Equals(Department, StringComparison.OrdinalIgnoreCase)).ToList();
        if (FilteredUsers.Count > 0)
        {
            return Ok(FilteredUsers);
        }
        return NotFound($"No Users Found with Department: {Department}");
    }
}

```

As you can see in the above code, we have marked the Department parameter with the FromQuery attribute, which means the ASP.NET Core Framework will now map this parameter with the query string parameter named Department. Now, run the application and try to access the above endpoint using the Department query string as shown in the below image:



Customizing Query Parameter Names:

We can customize the query parameter names by specifying the Name Property in the [FromQuery] attribute if the query string parameter name is different from the method parameter name. For a better understanding, please modify the User Controller as follows:

```
using Microsoft.AspNetCore.Mvc;
```

```

using ModelBinding.Models;
namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UserController : ControllerBase
    {
        private static List<UserModel> Users = new List<UserModel>
        {
            new UserModel { Id = 1, Name = "Rakesh", Department = "IT", Gender =
"Male", Salary = 1000 },
            new UserModel { Id = 2, Name = "Priyanka", Department = "IT", Gender
= "Female", Salary = 2000 },
            new UserModel { Id = 3, Name = "Suresh", Department = "HR", Gender =
"Male", Salary = 3000 },
            new UserModel { Id = 4, Name = "Hina", Department = "HR", Gender =
"Female", Salary = 4000 },
            new UserModel { Id = 5, Name = "Pranaya", Department = "HR", Gender =
"Male", Salary = 35000 },
            new UserModel { Id = 6, Name = "Pooja", Department = "IT", Gender =
"Female", Salary = 2500 },
        };

        [HttpGet]
        public IActionResult GetProducts([FromQuery(Name = "Dept")] string
Department)
        {
            // Implementation to retrieve employees based on the Department
            var FilteredUsers = Users.Where(emp =>
emp.Department.Equals(Department, StringComparison.OrdinalIgnoreCase)).ToList();
            if (FilteredUsers.Count > 0)
            {
                return Ok(FilteredUsers);
            }
            return NotFound($"No Users Found with Department: {Department}");
        }
    }
}

```

With the above changes in place, you need to use the query string name Dept instead of Department, as shown in the image below.

```
localhost:7121/api/User?Dept=HR x +
localhost:7121/api/User?Dept=HR
[{"id":3,"name":"Suresh","department":"HR","gender":"Male","salary":3000},
{"id":4,"name":"Hina","department":"HR","gender":"Female","salary":4000},
{"id":5,"name":"Pranaya","department":"HR","gender":"Male","salary":35000}]
```

When Should We Use the Name Property of the FromQuery Attribute in ASP.NET Core Web API?

The Name property of the FromQuery attribute is useful when the query string parameter name does not match the parameter name in your action method. For example, if the query string contains a parameter named Dept but you want to bind it to a method parameter named DepartmentId, you can specify [FromQuery(Name="Dept")] int DepartmentId to map these names correctly. This is useful when:

- The query string parameter name does not match the action method parameter name.
- You want to provide more descriptive or user-friendly names for the query string parameters.

FromQuery with Complex Type in ASP.NET Core Web API:

We can use the FromQuery attribute with complex types when we want to bind multiple query string parameters to the properties of a complex type. The query string keys must match the property names of the complex type. Let us understand this with an example. Suppose we want to search the users by Department and Gender. So, let us first create a class file named **UserSearch.cs** and then copy and paste the following code. We will use this model to map with the query string parameters.

```
namespace ModelBinding.Models
{
    public class UserSearch
    {
        public string Department { get; set; }
        public string Gender { get; set; }
    }
}
```

Modifying the User Controller:

Next, modify the User Controller as follows. Here, we have marked the UserSearch parameter with the FromQuery Attribute, meaning its properties will map to the query string parameter.

```
using Microsoft.AspNetCore.Mvc;
```

```

using ModelBinding.Models;
namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UserController : ControllerBase
    {
        private static List<UserModel> Users = new List<UserModel>
        {
            new UserModel { Id = 1, Name = "Rakesh", Department = "IT", Gender =
"Male", Salary = 1000 },
            new UserModel { Id = 2, Name = "Priyanka", Department = "IT", Gender
= "Female", Salary = 2000 },
            new UserModel { Id = 3, Name = "Suresh", Department = "HR", Gender =
"Male", Salary = 3000 },
            new UserModel { Id = 4, Name = "Hina", Department = "HR", Gender =
"Female", Salary = 4000 },
            new UserModel { Id = 5, Name = "Pranaya", Department = "HR", Gender =
"Male", Salary = 35000 },
            new UserModel { Id = 6, Name = "Pooja", Department = "IT", Gender =
"Female", Salary = 2500 },
        };

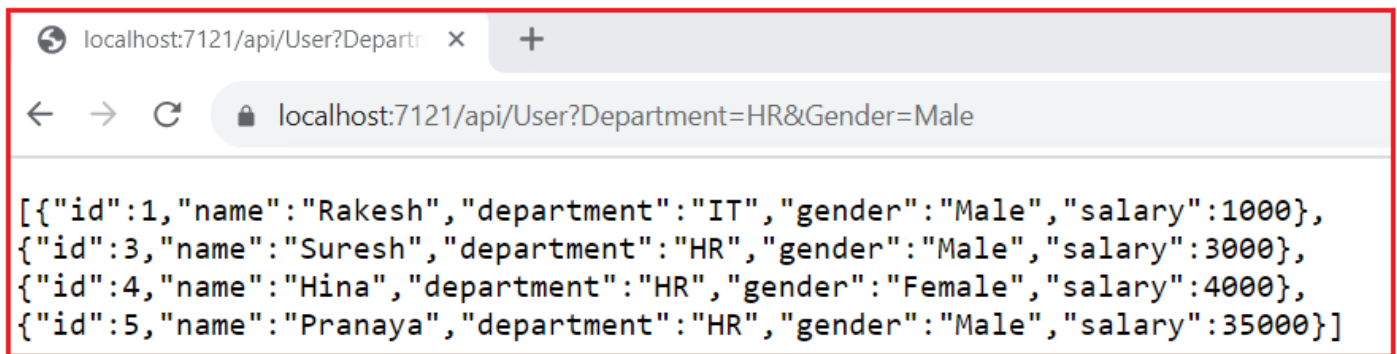
        [HttpGet]
        public IActionResult GetUsers([FromQuery] UserSearch userSearch)
        {
            var FilteredUsers = new List<UserModel>();
            if (userSearch != null)
            {
                FilteredUsers = Users.Where(
                    emp => emp.Department.Equals(userSearch.Department,
StringComparison.OrdinalIgnoreCase) ||
                    emp.Gender.Equals(userSearch.Gender,
StringComparison.OrdinalIgnoreCase)
                ).ToList();

                if (FilteredUsers.Count > 0)
                {
                    return Ok(FilteredUsers);
                }
                return NotFound($"No Users Found with Department:
{userSearch?.Department} and Gender: {userSearch?.Gender}");
            }
            return BadRequest("Invalid Search Criteria");
        }
    }
}

```

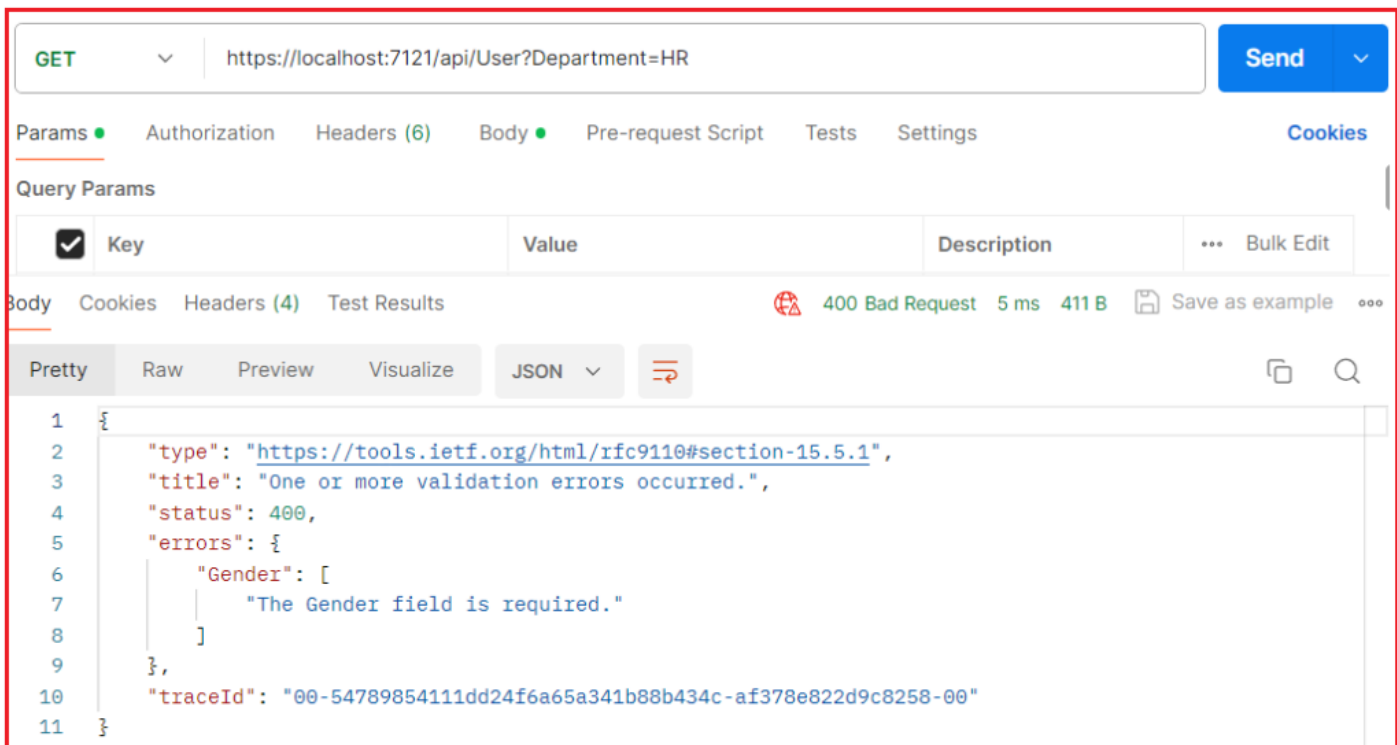
In this case, both the Department and Gender properties of the UserSearch object will be bound from the query string parameters **Department** and Gender. This is useful when you have multiple related query string parameters that can logically be grouped into a single object.

With the above changes in place, run the application and access the above endpoint with the Department and Gender Query string, such as **Department=HR&Gender=Male**. It should work as expected, as shown in the image below. For a query string **Department=HR&Gender=Male**, ASP.NET Core will create an instance of **UserSearch** with **Department** set to "HR" and **Gender** set to **Male**.



```
[{"id":1,"name":"Rakesh","department":"IT","gender":"Male","salary":1000}, {"id":3,"name":"Suresh","department":"HR","gender":"Male","salary":3000}, {"id":4,"name":"Hina","department":"HR","gender":"Female","salary":4000}, {"id":5,"name":"Pranaya","department":"HR","gender":"Male","salary":35000}]
```

In the above example, both Department and Gender query string parameters are mandatory. For example, if you only pass the Department query string, then you will get the following error:



```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Gender": [
      "The Gender field is required."
    ]
  },
  "traceId": "00-54789854111dd24f6a65a341b88b434c-af378e822d9c8258-00"
}
```

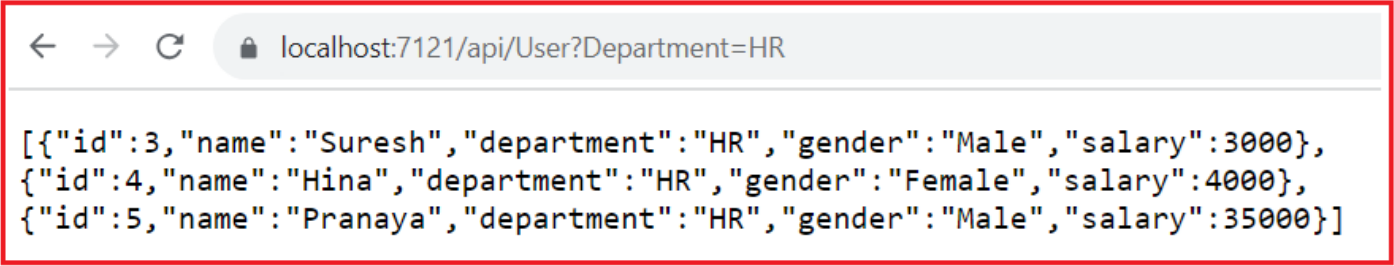
This is because, in our UserSearch model class, we have not made the Gender and Department nullable type. We have also decorated our User Controller with the ApiController attribute. The ApiController will consider all non-nullable types as mandatory fields, and if no value is supplied for those parameters, then it will return 400 Bad Requests, and that is what you see in the previous request.

How Do We Make the Query String Parameters Optional in ASP.NET Core Web API?

If you want to make the Query string parameters optional, then you need to make the action method parameters optional. So, what we need to do is, we need to make the Gender and Department parameters nullable type. So, modify the UserSearch class as follows:

```
namespace ModelBinding.Models
{
    public class UserSearch
    {
        public string? Department { get; set; }
        public string? Gender { get; set; }
    }
}
```

With the above changes, run the application and access the endpoint, and you should get the result as expected, as shown in the below image:



← → ↻ 🔒 localhost:7121/api/User?Department=HR

```
[{"id":3,"name":"Suresh","department":"HR","gender":"Male","salary":3000},
{"id":4,"name":"Hina","department":"HR","gender":"Female","salary":4000},
{"id":5,"name":"Pranaya","department":"HR","gender":"Male","salary":35000}]
```

When Should We Use the FromQuery Attribute in ASP.NET Core Web API?

FromQuery Attribute is used when we need to capture simple data sent in the query string of a GET request. It is ideal for filtering data requests, paging requests, or other scenarios where the client needs to pass optional or additional data to the server without using the URL path. This is useful when:

- We want to override the default binding source (e.g. if the parameter name could be confused with a route parameter or a form value).
- We have multiple possible sources for a parameter and need to choose the query string explicitly.
- We want to support optional query string parameters.

In the next article, I will discuss [Model Binding Using FromRoute in ASP.NET Core Web API](#) with Examples. In this article, I will try to explain **model binding using from query in ASP.NET Core Web API** with examples. I hope you enjoy this article, “Model Binding Using FromQuery in ASP.NET Core Web API.”

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

[Consent](#)[Do Not Sell My Information](#)